

- 2) Delete a record from the database
- 3) Commit the transaction

```
testdb=# BEGIN;                                # Step-1

testdb=# DELETE FROM student_table WHERE id = 1; # Step-2

testdb=# COMMIT;                                # Step-3
```

Import and export databases

We can export table data into a CSV file. The example given below shows this.

To export the entire table, execute the following command:

```
testdb=# COPY student_table TO '/tmp/student_table.csv'
DELIMITER ',' CSV HEADER;
```

We can also export only certain columns from the table.

For instance, the command given below exports name and age columns only:

```
testdb=# COPY student_table(name, age) TO '/tmp/student_
table.csv' DELIMITER ',' CSV HEADER;
```

We can export a table without a header, i.e., column names. To achieve this, exclude the HEADER keyword in the SQL command, as shown below:

```
testdb=# COPY student_table TO '/tmp/student_table.csv'
DELIMITER ',' CSV;
```

In addition to this, we can also import data from CSV files. The statement below shows this:

```
testdb=# COPY student_table FROM '/tmp/student_table.csv'
DELIMITER ',' CSV HEADER;
```

Python language interface

PostgreSQL supports programming interfaces in C/C++, Java, Python, Perl and PHP. Using these interfaces, we can perform operations on databases. This section discusses the Python language interface. PostgreSQL can be accessed in Python using the `psycopg2` module. To install it, execute the following command in a terminal:

```
$ sudo apt-get install python-psycopg2
```

Now, let us write a Python program to display the table of contents:

```
#!/usr/bin/python
import psycopg2
# Step-1: Establish connection with DB
conn = psycopg2.connect(database = "testdb",
```

```
user = "postgres",
password = "my-password",
host = "127.0.0.1",
port = "5432")
```

```
# Step-2: Create a cursor to perform operation on DB
cur = conn.cursor()
```

```
# Step-3: Execute select query
cur.execute("SELECT id, name from student_table")
cnt=1
rows = cur.fetchall()
for row in rows:
    print "Record-%d:" %cnt
    print "\tID : ", row[0]
    print "\tNAME: ", row[1]
    cnt += 1
```

```
# Step-4: Close DB connection
conn.close()
```

Note that this program doesn't modify the database; hence, the commit operation is not required. If any operation modifies the database, like *INSERT*, *UPDATE*, *DELETE* and so on, then we have to execute the *commit()* method after modification; otherwise, the changes won't be reflected in the database. The Python program given below shows the usage of the commit method:

```
#!/usr/bin/python
```

```
import psycopg2
```

```
# Step-1: Establish connection with DB
conn = psycopg2.connect(database = "testdb",
    user = "postgres",
    password = "my-password",
    host = "127.0.0.1",
    port = "5432")
```

```
# Step-2: Create a cursor to perform operation on DB
cur = conn.cursor()
```

```
# Step-3: Execute insert query
cur.execute('INSERT INTO student_table VALUES(
    4, 'Twitty', 20, 'Venus', 'CS')
```

This article has just given a few tips on how best to use PostgreSQL. There is a lot more that can be done with this excellent database. 

By: Narendra K.

The author is a FOSS enthusiast. He can be reached at narendra0002017@gmail.com.